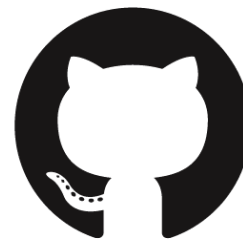




git &

GitHub

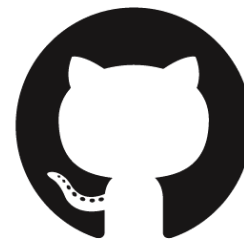


Web Programming I



git &

GitHub



Student is able to

use version control with Git and collaborative workflows with GitHub in web development projects



Content

VERSION CONTROL

- What is the purpose of version control?

LOCAL WORKFLOW REFRESHER

- What commands make up the essential local workflow?

- What makes a good commit message?

REMOTE ON GITHUB

- How is GitHub different from Git itself?

- How to work with remote repositories?

COLLABORATION AND BRANCHING

- How to work on multiple features simultaneously without conflicts?

- What is the difference between clone and fork?

PROBLEM-SOLVING

- How to recover from common Git problems?



VERSION CONTROL

What is the purpose of version control?

LOCAL WORKFLOW REFRESHER

What commands make up the essential local workflow?

What makes a good commit message?

REMOTE ON GITHUB

How is GitHub different from Git itself?

How to work with remote repositories?

COLLABORATION AND BRANCHING

How to work on multiple features simultaneously without conflicts?

What is the difference between clone and fork?

PROBLEM-SOLVING

How to recover from common Git problems?

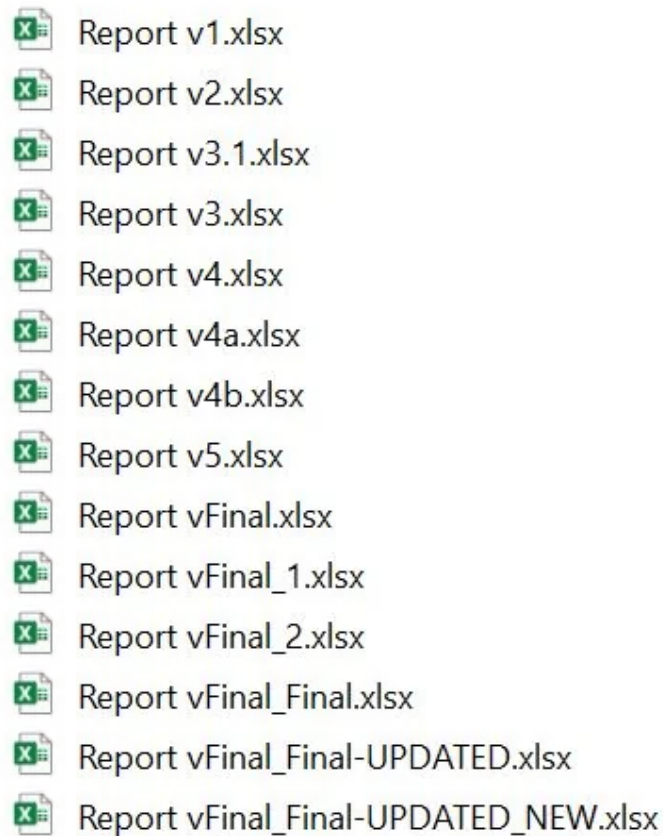


Version control > We want to avoid

We want to avoid

a) multiple versions in different files

b) overwriting the same file with no history





Git repository tracks project changes

- created by Linus Torvalds in 2005 to manage the development of Linux Kernel
- git repository is a **.git/** folder inside a project (folder)
- tracks all changes made to files, building a history over time



GitHub profile as portfolio



★ 3,586 reads

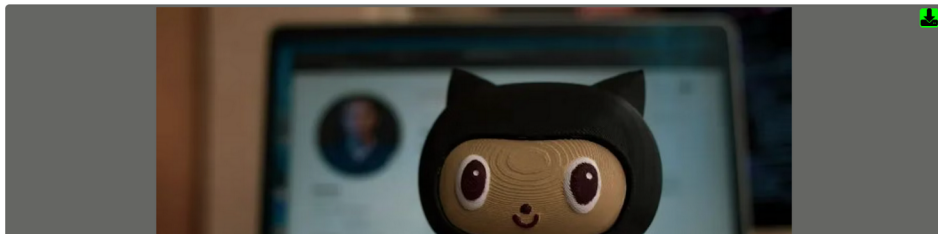
A Developer's Github Profile is Just as Important as their Resume Itself

by Lane Wagner • May 8th, 2022



TECH

EN ES HI ZH VI FR PT JA



What to Expect in an Interview

In the latter interview rounds, engineers and hiring managers may look through your projects in greater detail. This entails looking through the README and code on GitHub project repos. For certain interviews, candidates are asked to walk the interviewers through the project and justify design decisions. Interviewers may also check for best practices around project structure, app design, testing, commits, etc.

Having a solid GitHub profile and creating good READMEs for projects helps candidates in the application stage and may also help them during the latter interview rounds. There's no downside to spending some time sprucing up your GitHub profile and project overviews.

Let's look at some concrete ways to improve both your GitHub profile and your Git commits.

GitHub Profile Best Practices

The goal of your GitHub profile during a job search is to showcase your best projects, skills, and coding history to potential employers. Here are some guidelines that will help you create a unique profile that highlights the value you can bring to a team:



VERSION CONTROL

What is the purpose of version control?

LOCAL WORKFLOW REFRESHER

What commands make up the essential local workflow?

What makes a good commit message?

REMOTE ON GITHUB

How is GitHub different from Git itself?

How to work with remote repositories?

COLLABORATION AND BRANCHING

How to work on multiple features simultaneously without conflicts?

What is the difference between clone and fork?

PROBLEM-SOLVING

How to recover from common Git problems?



Installation and initialization

Ubuntu:

```
$ sudo apt install git
```

Mac:

- Xcode
- Home Brew

Windows:

- Git SCM: <https://git-scm.com>

```
$ mkdir mynewproject # Create a new  
folder for our project
```

```
$ cd mynewproject # navigate to  
project folder
```

```
$ git init
```

Now our project folder is a Git repository.



Three sections and states of Git

Working
Directory

Where we
edit files

Staging
Area

Where you prepare
changes for the next
commit.

Local
Repository

The code that has
been committed

Remote
Repository

The remote server
for sharing and
collaboration

----- Three sections of a Git project -----

A file in the working directory can be unmodified, modified, or staged. The staging area is where staged changes are recorded. The Git directory stores committed snapshots



Core workflow > \$ git status

\$ git status

Working
Directory

Staging
Area

Local
Repository

Remote
Repository

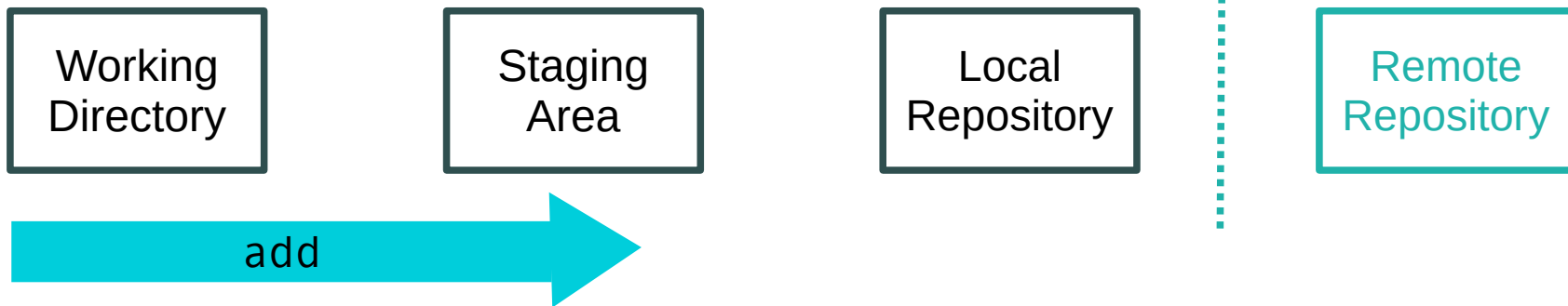
status

Use `git status` to display the state of the working directory and the staging area



Core workflow > \$ git add

\$ git add

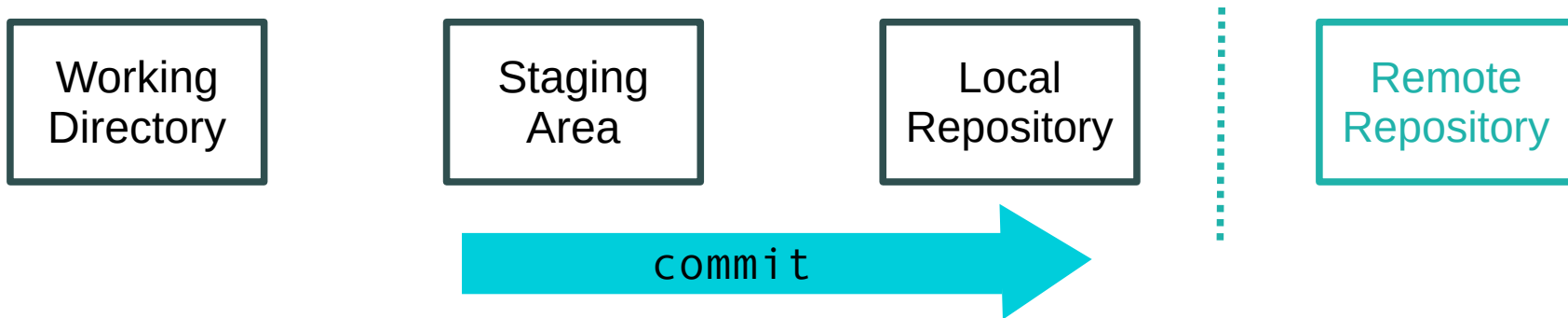


Use `git add` to move a file to the staging area.



Core workflow > \$ git commit

\$ git commit



Use `git commit -m "My custom message"` to save a snapshot of all staged changes to the local repository. This file is now tracked and a copy of this version is kept going forward.

Use `git log` or `git log --oneline` to see your commit history. Each commit has a unique identifier (SHA-1).

The message that you specify will help you easily identify this specific version.



Commit naming convention

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

```
→ ng-poopy master ✗ git add .  
→ ng-poopy master ✗ git cz
```

All commit message lines will be cropped at 100 characters.

? Select the type of change that you're committing: (Use arrow keys)

```
> feat: A new feature  
fix: A bug fix  
docs: Documentation only changes  
style: Changes that do not affect the meaning of the code  
(white-space, formatting, missing semi-colons, etc)  
refactor: A code change that neither fixes a bug or adds a feature  
perf: A code change that improves performance  
test: Adding missing tests  
chore: Changes to the build process or auxiliary tools  
and libraries such as documentation generation
```



Excluding files from version control

By default, `git add .` will stage all files in the directory — including ones you may not want tracked. If we want to prevent some files from being tracked and potentially made public later on, we add them to the **.gitignore** file.

Once a file is tracked, `.gitignore` alone won't help — you'll need to untrack it manually.

Example content of
.gitignore file

```
*.pem  
*.log  
passwords.txt
```



Core workflow > Workflow recap: add → commit

Workflow recap: add → commit

We initialize our git project

```
$ git init
```

Create .gitignore for files you don't want to track

We create and work on our files

We “freeze” a first a version of our project

```
$ git add . # This stages all files
```

We commit the staged files and name the commit

```
$ git commit -m “First version”
```



User Interfaces

- ▶ CLI command line
- ▶ VS Code Git integration
- ▶ GitHub Desktop

```
merge-git-playground > target.js > printMessage

Incoming 7b18bdb - theirs
13
14 /**
15  * Prints the welcome message
16  */
17 function printMessage(showUsage, showVersion) {
18   console.log("Welcome To Line Counter");
19   if (showVersion) {
20     console.log("Version: 1.0.0");
21   }
22   if (showUsage) {
23     console.log("Usage: node base.js <file1>");
24   }
}

Current b7bd9b1 - main
13
14 /**
15  * Prints the welcome message
16  */
17 function printMessage(showUsage, message) {
18   console.log(message);
19 }
20 if (showUsage) {
21   console.log("Usage: node base.js <file1>");
22 }

Result merge-git-playground\target.js 1 Conflict Remaining
13
14 /**
15  * Prints the welcome message
16  */
17 function printMessage(showUsage) {
18   console.log("Welcome To Line Counter");
19 }
20 if (showUsage) {
21   console.log("Usage: node base.js <file1> <file2> ...");
22 }

Complete Merge
```



VERSION CONTROL

What is the purpose of version control?

LOCAL WORKFLOW REFRESHER

What commands make up the essential local workflow?

What makes a good commit message?

REMOTE ON GITHUB

How is GitHub different from Git itself?

How to work with remote repositories?

COLLABORATION AND BRANCHING

How to work on multiple features simultaneously without conflicts?

What is the difference between clone and fork?

PROBLEM-SOLVING

How to recover from common Git problems?



Remote on GitHub > Git and the platforms

Git and the platforms

- git – version control locally
- GitHub – a cloud-based platform for hosting and collaboration



BitBucket <https://bitbucket.org>



Codeberg

Codeberg <https://codeberg.org/>



GitHub <https://github.com>



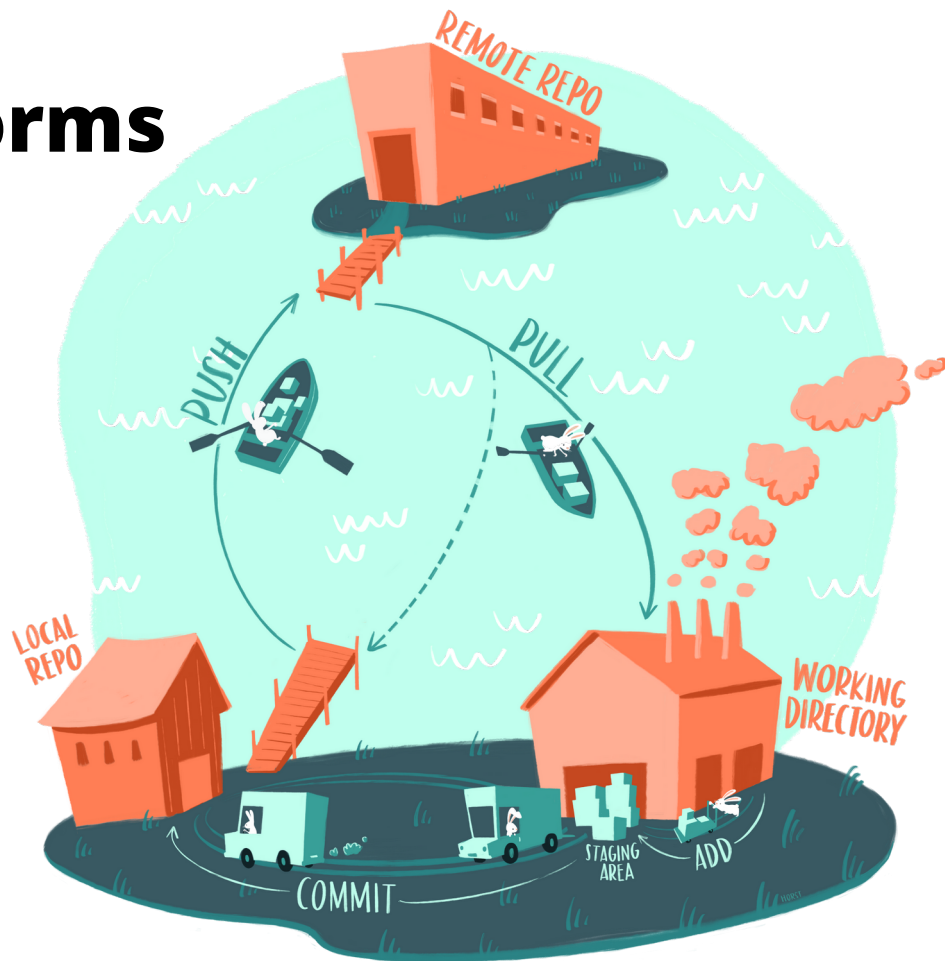
GitLab <https://about.gitlab.com>

Working
Directory

Staging
Area

Local
Repository

Remote
Repository





Configuring GitHub account locally

Author local commits with username from GitHub

```
$ git config --global user.email "you@example.com"
```

```
$ git config --global user.name "Your Name"
```

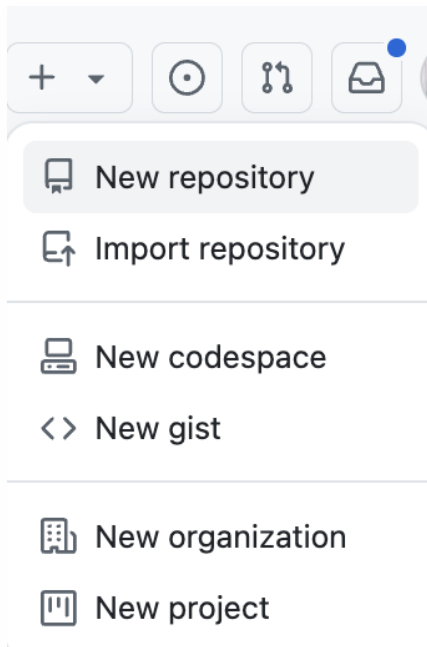
Priority:

1. --local (project specific) – local bypasses global in the folder
2. --global (user specific)
3. --system (whole system)



Remote on GitHub > Creating a new GitHub repo

New GitHub repo



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * sonsoleslp / Repository name * wohi-test
✔ wohi-test is available.

Great repository names are short and memorable. Need inspiration? How about **refactored-octo-bassoon** ?

Description (optional)

- ☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

- ☐ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: **None**

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: **None**

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

ⓘ You are creating a public repository in your personal account.

Create repository



Synchronize with local repository

When we
don't have
an existing local
repository yet



...or create a new repository on the command line

```
echo "# kazhigram" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/kuisdemo/kazhigram.git
git push -u origin main
```



When we
already have
an existing local
repository



...or push an existing repository from the command line

```
git remote add origin https://github.com/kuisdemo/kazhigram.git
git branch -M main
git push -u origin main
```





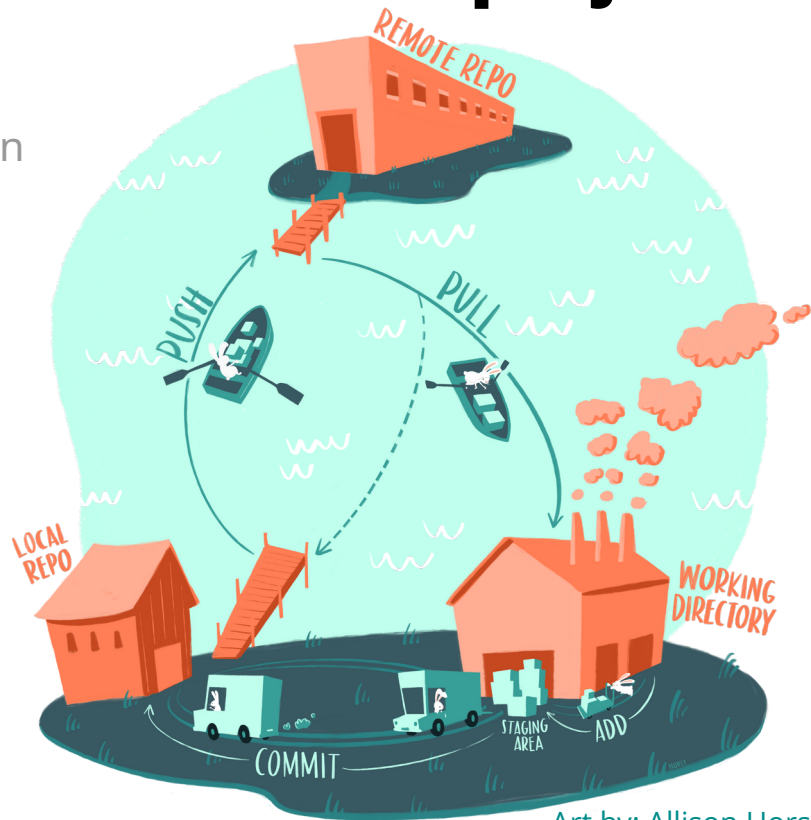
Remote on GitHub > Linking a local project to a remote project

Linking a local project to a remote project

In a directory where git init has been already run

```
$ git remote add origin  
https://github.com/your-username/your-repo-name.git
```

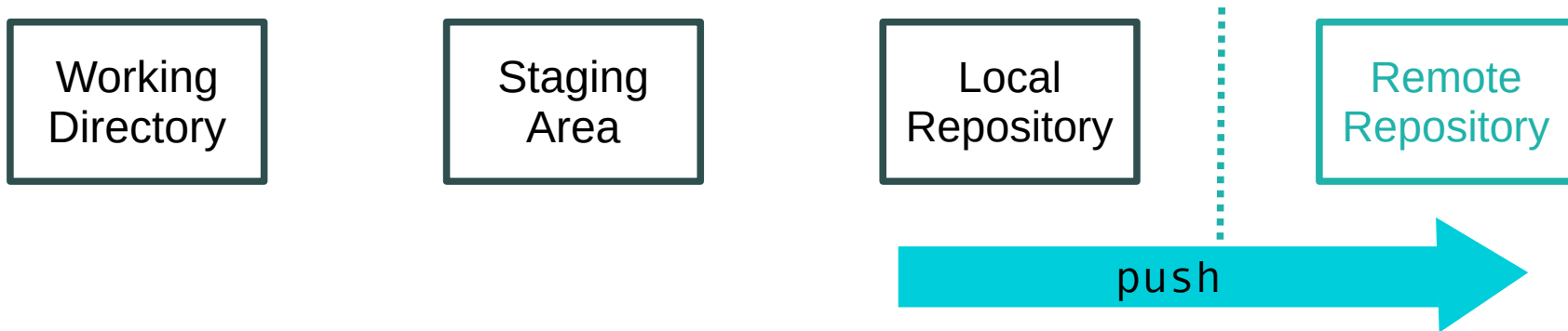
Now git pull/push commands will synchronize our work between the local and remote repositories





Remote on GitHub > `$ git push -u origin main`

\$ git push -u origin main

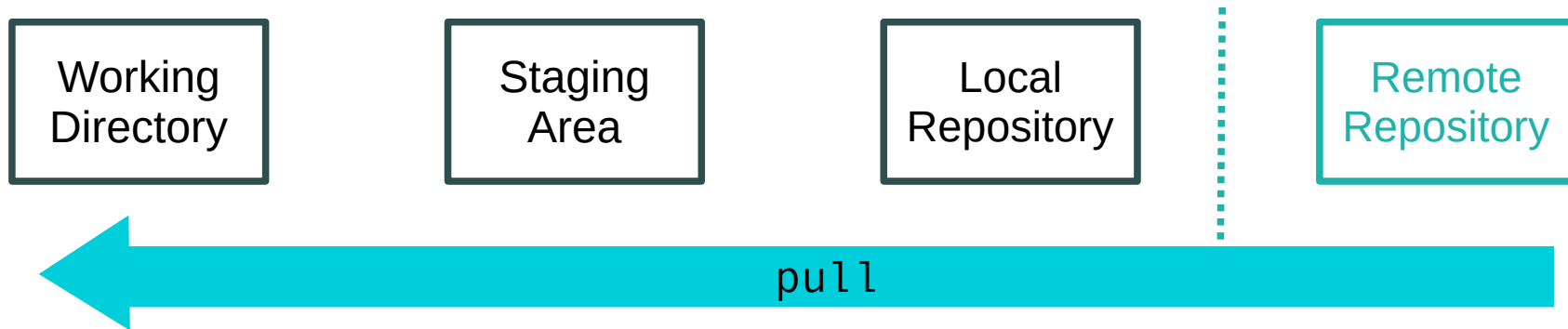


Use `git push` to send committed changes from your current branch to the remote repository.



Remote on GitHub > \$ git pull

\$ git pull



Use `git pull` to get data from remote repo.

`git pull` = `git fetch` + `git merge`

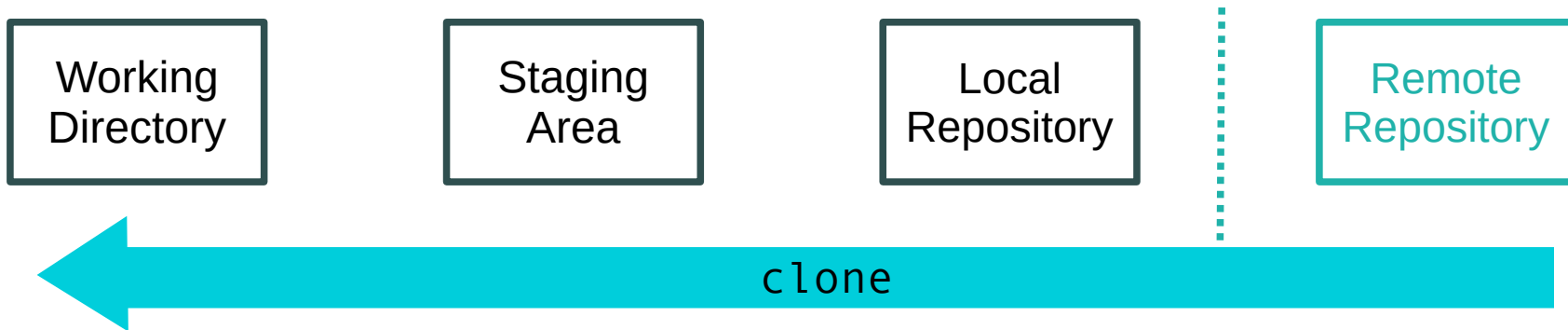
`fetch` downloads remote changes to your local repo, without changing your working files

`pull` fetches and then merges



Remote on GitHub › Downloading an existing repository

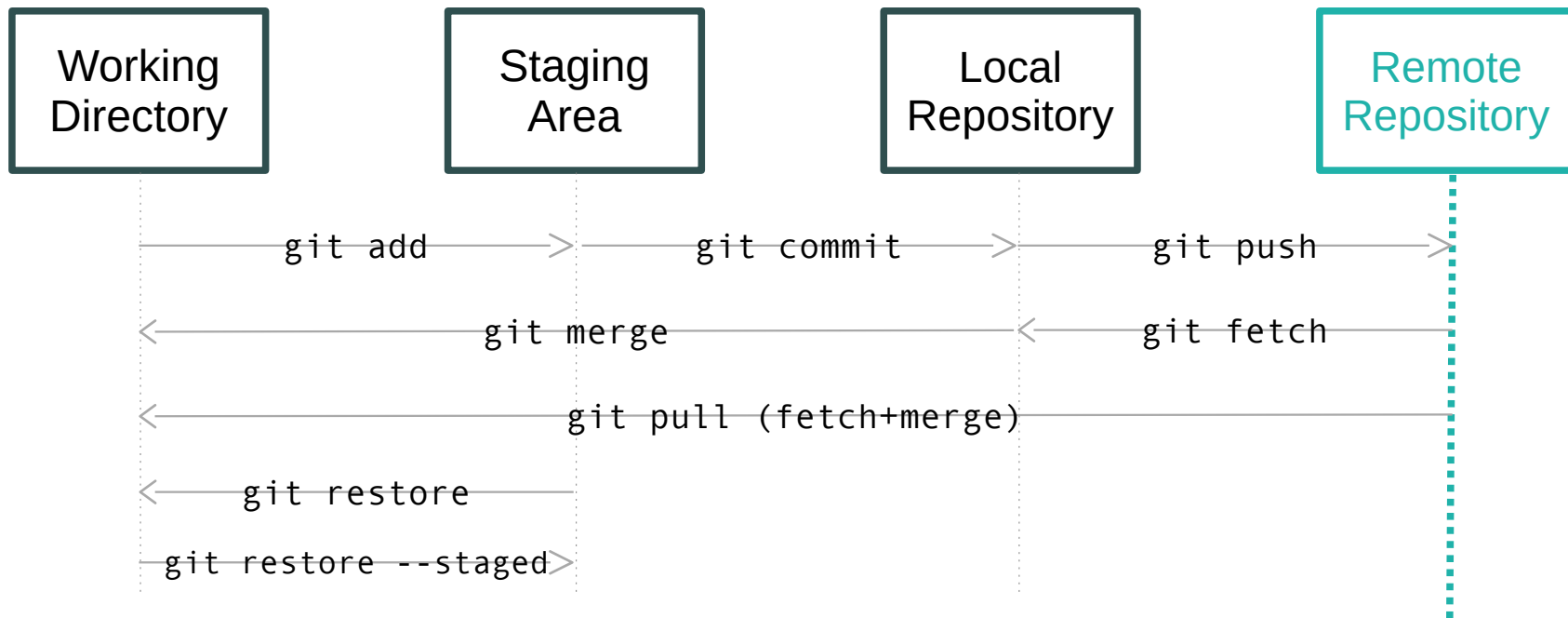
Downloading an existing repository



```
$ git clone https://github.com/twbs/bootstrap.git
```



Moving changes between Git's areas





Remote on GitHub > Publishing with GitHub Pages

Publishing with GitHub Pages

Make your files stored in the GitHub repository accessible as a webpage rather than just “files” with GitHub Pages

The website will be available at
myuser.github.io/myrepository

<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings

GitHub Pages source saved.

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Code security

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch

Branch

Your GitHub Pages site is currently being built from the main branch. [Learn more about configuring the publishing source for your site.](#)

main / (root) Save

Learn how to [add](#)

Custom domains

Custom domains [Learn more about configuring custom domains.](#)

/docs

☒ Enforce HTTPS

HTTPS provides a layer of encryption that prevents others from snooping on or tampering with traffic to your site. When HTTPS is enforced, your site will only be served over HTTPS. [Learn more about securing your GitHub Pages site with HTTPS.](#)

Visibility [GitHub Enterprise](#)

With a GitHub Enterprise account, you can restrict access to your GitHub Pages site by publishing it privately. You can use privately published sites to share your internal documentation or knowledge base with members of your enterprise. You can try GitHub Enterprise risk-free for 30 days. [Learn more about the visibility of your GitHub Enterprise site.](#)



Tagging versions

A tag marks a specific commit as a named version.

Use tags to mark releases, milestones, or stable versions.

Create a tag:

```
git tag v1-html
```

Push a tag to GitHub:

```
git push origin v1-html
```

List all tags:

```
git tag
```

Tags appear under "Tags" in your GitHub repository.



GitHub Releases

A Release is a tag + description + downloadable files.
It gives users a clear way to download a specific version.

How to create a Release:

1. Go to your repository on GitHub
2. Click Releases → Create a new release
3. Choose a tag (or create one)
4. Add a title and description
5. Publish release

Users can download any release as a ZIP file without needing Git.
→ useful for sharing specific versions of your project



VERSION CONTROL

What is the purpose of version control?

LOCAL WORKFLOW REFRESHER

What commands make up the essential local workflow?

What makes a good commit message?

REMOTE ON GITHUB

How is GitHub different from Git itself?

How to work with remote repositories?

COLLABORATION AND BRANCHING

How to work on multiple features simultaneously without conflicts?

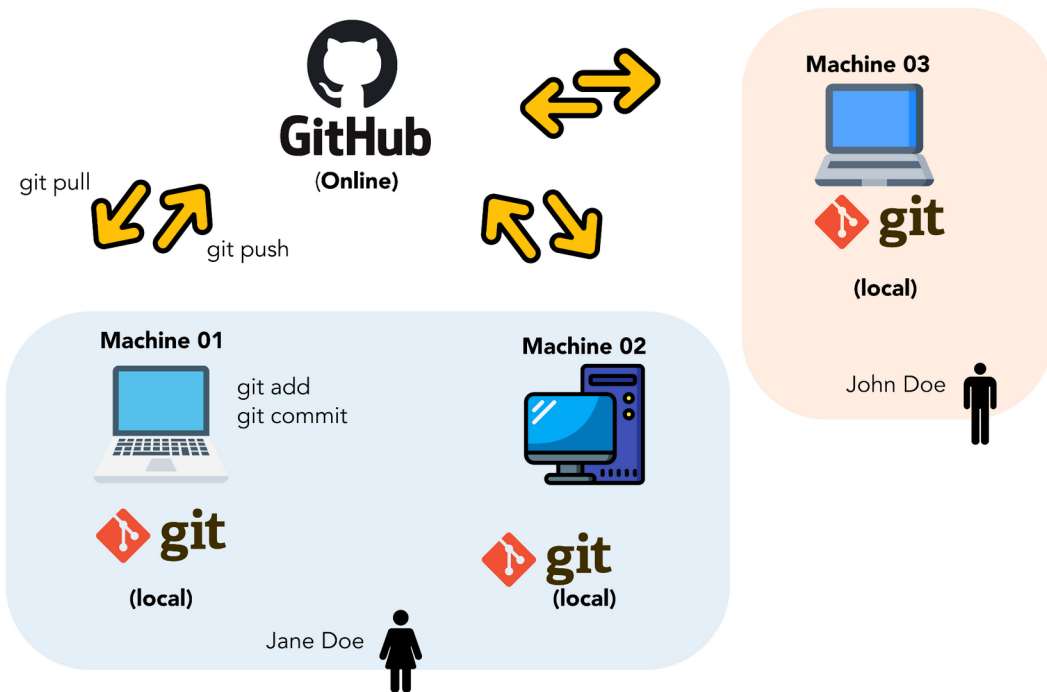
What is the difference between clone and fork?

PROBLEM-SOLVING

How to recover from common Git problems?



Collaboration in remote repos





Adding collaborators

<> Code Issues Pull requests Actions Projects Wiki Security Insights **Settings** 1

General

Access

Collaborators 2

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Code security

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Who has access

Public repository

This repository is public and visible to anyone.

PUBLIC REPOSITORY

This repository is public and visible to anyone.

[Manage](#)

DIRECT ACCESS

0 collaborators have access to this repository. Only you can contribute to this repository.

Manage access

You haven't invited any collaborators yet

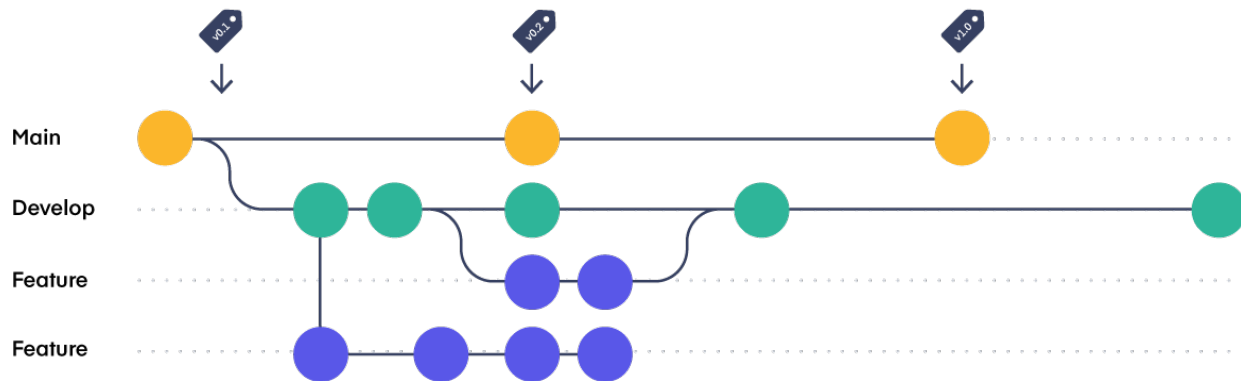
Add people

 3



Branching enables isolated feature work

- Isolated feature work
- Safer than working directly on main
- Enables collaboration without overwriting each other's work





Basic branch commands

```
git branch # listing existing branches
```

```
git branch name_of_branch # creating a new branch
```

```
git switch name_of_branch # switching to a branch
```

Shortcut: create and switch in one command:

```
git switch -c name_of_branch
```

```
git branch -d name_of_branch # deleting (clean up after merge)
```

Note: In older tutorials you'll see git checkout instead of git switch. They do the same thing for branches, but switch is clearer because it only handles branch switching.

Modern Git (2.23+) splits the old checkout command:

git switch → for switching branches

git restore → for restoring files

git checkout → does both (older, still works)



Merging branches

1. Create branch and switch to it

```
$ git switch -c name_of_branch
```

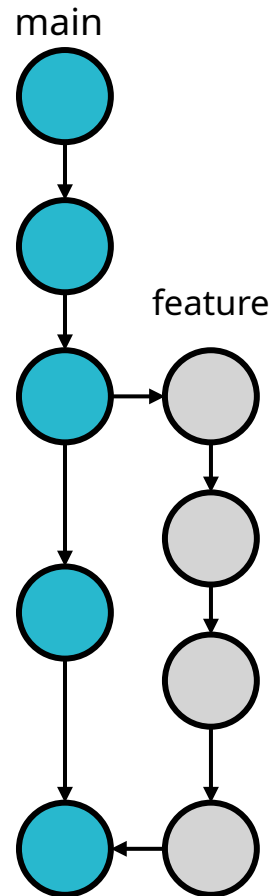
2. Edit files, git add, git commit, etc.

3. When ready to integrate, switch to main

```
$ git switch main
```

4. Merge the feature branch into main

```
$ git merge name_of_branch
```





Clone vs. Fork – the difference

- **clone** copies an existing repo to your own computer, makes a local copy
- Use clone when you want to
 - Get a local working copy of a repository
 - Work on a repo where you already have access
 - Download your own repo to your computer
- **fork** makes a remote copy of someone's repo under your own account
- Use fork when
 - You don't have permission to push directly to the original repo
 - You want your own writable copy on GitHub
 - You want to contribute changes back using a pull request



Pull request workflow

fork → branch → commit & push → pull request → review → merge

1. **Fork** the project to your own GitHub account
2. **Clone** your fork
3. Create **branch** for your changes
4. **Commit and push** your work to your fork
5. **Open a pull request** against the original repo

6. **Review** and discussion: maintainers can comment and request modifications
7. **Merge**: maintainers merge





VERSION CONTROL

What is the purpose of version control?

LOCAL WORKFLOW REFRESHER

What commands make up the essential local workflow?

What makes a good commit message?

REMOTE ON GITHUB

How is GitHub different from Git itself?

How to work with remote repositories?

COLLABORATION AND BRANCHING

How to work on multiple features simultaneously without conflicts?

What is the difference between clone and fork?

PROBLEM-SOLVING

How to recover from common Git problems?



Problem-solving mindset

When something goes wrong:

1. Run `git status` — always start here
2. Identify your current branch
3. Ask: is the problem local, branch-related, or remote?
4. Read the error message carefully
5. Do not force-push or delete anything unless you know why



Problem: Merge conflict

What happened: You tried to merge a branch into another (e.g., feature into main), and Git stopped with a conflict message. The conflicted file shows markers like <<<<<<, =====, and >>>>>>.

Why it happens: Two branches changed the same lines in the same file differently. Git cannot decide which version to keep, so it asks you.

- Conflicts are normal in collaborative work
- Don't leave conflict markers in the file (Git won't complete the merge)
- Don't force-push to overwrite the other branch's changes

What to do

1. Run `git status` — see which files have conflicts
2. Open the conflicted file
3. Look for markers <<<<<< HEAD and >>>>>> branch_name
4. Choose which version to keep (or combine both)
5. Remove all conflict markers
6. Save the file
7. `git add <filename>`
8. `git commit -m "resolve merge conflict"`



Problem: Push rejected (remote ahead)

What happened: You tried to push, but Git rejected it with a message like:

```
"! [rejected] main -> main (non-fast-forward)"
```

```
"Updates were rejected because the remote contains work that you do not have locally"
```

Why it happens: Someone else pushed to the same branch before you, or you made changes on GitHub directly (e.g., edited README.md online). Your local branch is now behind the remote.

- Don't use `git push --force` to overwrite the remote (you could delete others' work)
- Don't delete your local repo and start over

What to do

1. Fetch the remote changes:
`$ git pull`
2. Resolve any merge conflicts if they appear
3. Push your changes:
`$ git push`



Problem: Wrong branch

What happened: You made changes and committed them, but realized you were on the wrong branch (e.g., committed to main when you meant to commit to a feature branch).

Why it happens: It's easy to forget which branch you're on. Always check with git status before starting new work.

- Don't force-push to rewrite shared history
- Don't panic — if you catch it early (before push), it's easy to fix

What to do

If NOT yet pushed (only local):

```
$ git reset --soft HEAD~1 # undo last commit, changes stay  
                           in staging area
```

```
$ git stash # save changes temporarily
```

```
$ git switch correct-branch
```

```
$ git stash pop # bring changes back
```

```
$ git add .
```

```
$ git commit -m "your message"
```

If already pushed (already on GitHub):

```
$ git switch -c new-branch # create new branch from  
                           current state
```

```
$ git push -u origin new-branch # push new branch to GitHub
```

```
$ git switch main # switch back to main
```

```
$ git log --oneline # check the commit id from log
```

```
$ git revert <commit-id> # undo the wrong commit on main
```

```
$ git push # share the fix
```



Problem: Detached HEAD

What happened: Git shows a message like: "You are in 'detached HEAD' state. You can look around, make experimental changes and commit them..."

Why it happens: You checked out a specific commit (e.g., `git checkout <commit-hash>`) instead of a branch (`git checkout <branch-name>`). You're not on any branch — you're floating at a specific commit.

- Don't commit new work while in detached HEAD state without creating a branch first (your commits could become "lost"; unreferenced and eventually garbage-collected)

What to do

If you just wanted to look at the old version:

```
$ git switch main # return to  
main branch
```

If you want to keep changes made in detached HEAD:

```
$ git switch -c new-branch-  
name # create a new branch  
here
```



Problem: Accidentally tracked files

What happened: You committed and possibly pushed files that shouldn't be tracked in version control.

Why it happens: You ran `git add .` without a `.gitignore` file, or added the file before the `.gitignore` was set up.

Don't just delete the file locally — Git still tracks it.

Prevention is much easier than cleanup: Always set up `.gitignore` before your first commit.

What to do (basic cleanup for all situations):

1. Stop tracking the file (keep it locally):
`$ git rm --cached <filename>` or `$ git rm --cached -r <folder-name>`
2. Add to `.gitignore` to prevent it in the future, e.g.
`$ echo "node_modules/" >> .gitignore`
3. Commit the cleanup:
`$ git add .gitignore`
`$ git commit -m "chore: ignore node_modules"`
`$ git push`

Important: After basic cleanup the file is still in Git history: anyone can look at old commits and see it.

a) **If it's just clutter** (`node_modules`, build files, logs): The cleanup above is enough. History contains the files, but new versions don't.

b) **If it contained secrets you can change** (passwords, API keys): Rotate the secret immediately — change the password or API key. The old one is exposed in history, but no longer usable.

c) **If it contained sensitive information you cannot change** (personal data, private documents, etc.): The file must be removed from Git history. This is more complex: Use `git filter-repo` or `BFG Repo-Cleaner`. Force-push the rewritten history: `git push -force`. Notify the team — their local copies will need to be re-cloned. *Be aware: others may have already cloned the repo. The data may have spread beyond your control.*



Problem: Authentication fails

What happened: Git shows an authentication error when you try to push or pull, often something like: "remote: Support for password authentication was removed on ..."

Why it happens: GitHub no longer accepts your account password for Git operations. You need either a Personal Access Token (PAT) or SSH keys.

- Don't commit tokens or keys to your repository — treat them like passwords

What to do:

Option 1: Personal Access Token. 1. Go to GitHub → Settings → Developer settings → Personal Access Tokens → Generate new token. 2. Give it a name and select "repo" permissions. 3. Copy the token — you'll only see it once. 4. When Git prompts you for a password, paste the token instead. Your remote URL stays as HTTPS: `https://github.com/username/repo.git`

Option 2: SSH keys. 1. Generate an SSH key pair on your computer: `$ ssh-keygen -t ed25519 -C "your@email.com"`. 2. Copy the public key content: `$ cat ~/.ssh/id_ed25519.pub`. 3. Add the public key to GitHub: Settings → SSH and GPG keys → New SSH key. 4. If your remote is currently HTTPS, change it to SSH format: `$ git remote set-url origin git@github.com:username/repo.git`. Now git push/pull will use SSH — no password needed.

Option 3: VS Code Git integration. Just use VS Code's built-in Source Control panel. It handles authentication automatically the first time you push, opening a browser window to sign in to GitHub.



Summary

- version control avoids messy file copies and overwrites, and keeps a clear history of all changes
- essential commands: git init, git status, git add, git commit, and .gitignore for excluding files
- good commit messages describe what and why the change was made
- git is the version control tool, GitHub is a cloud platform for hosting repos and enabling collaboration
- use git push to send changes, git pull to receive them, git clone to download an existing repository
- clone creates a local copy on your computer, fork creates your own remote copy on GitHub for contributing back via pull requests
- branches enable isolated feature work: create a branch, commit changes, and merge back to main when ready
- when something goes wrong, start with git status, and use known recovery patterns for common problems (merge conflicts: edit and re-commit; push rejected: pull first; wrong branch: reset + stash)



Bibliography

- Materials of the course Computación en Red - Universidad Politécnica de Madrid
- BST 260 Introduction to Data Science <https://datasciencelabs.github.io/2023/03-git.html>
- GitHub Docs <https://docs.github.com/en>
- Chacon & Straub: ProGit <https://git-scm.com/book/en/v2>
- Learn Git Branching <https://learngitbranching.js.org/?locale=en>
- GeeksForGeeks <https://www.geeksforgeeks.org/git/git-tutorial/>
- W3Schools <https://www.w3schools.com/git/>